Universidad Nacional Autónoma de México

Facultad de Ingeniería

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

Lara Hernandez Angel Husiel - 320060829

Laboratorio de Microcomputadoras

Grupo: 06 - Semestre: 2026-2

Practica 5:

Control de actuadores con GPIO.

Profesor:

Ing. Moises Melendez Reyes

Fecha de Entrega:

27 de Marzo del 2026

1. Objetivo:

Reforzará las habilidades para programar y configurar las funciones GPIO para controlar motores de corriente directa, motores de pasos y servomotores a través del microcontrolador Raspberry Pi Pico. Estudiar la importancia de los amplificadores de potencia.

Actividad 1

De acuerdo a la distribución indicada en el circuito de la figura 5-10; realizar las acciones de control indicadas en la tabla 5-2. Conectar la fuente externa con 5V.

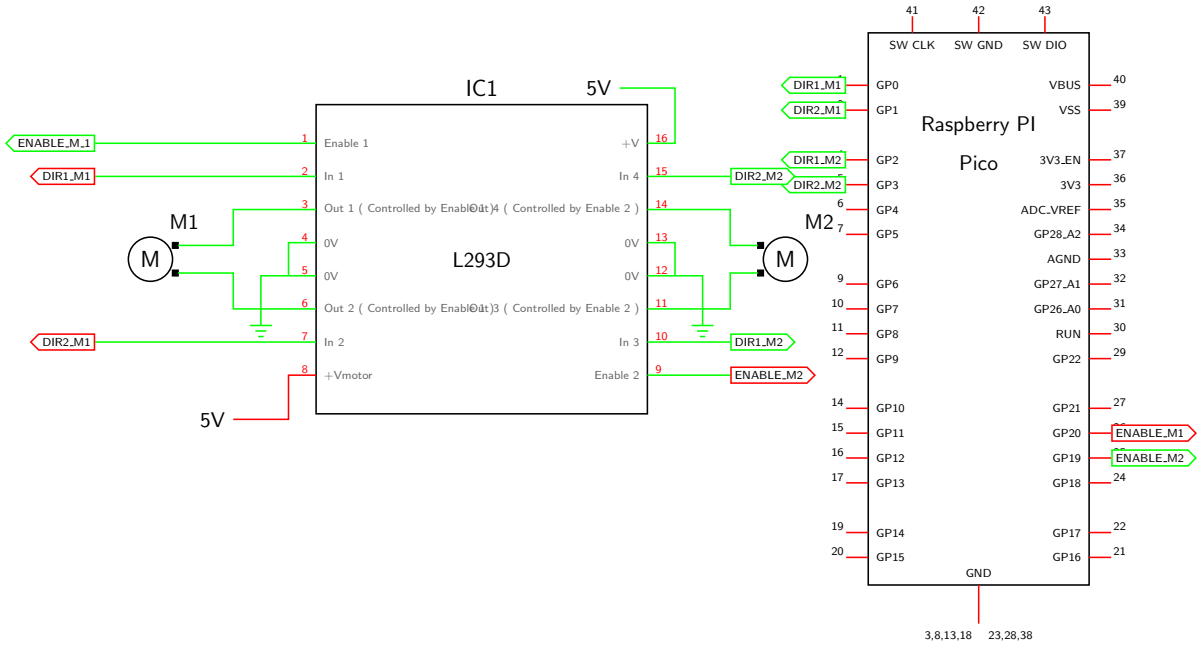


Figura 5-10. Control de motores de corriente directa

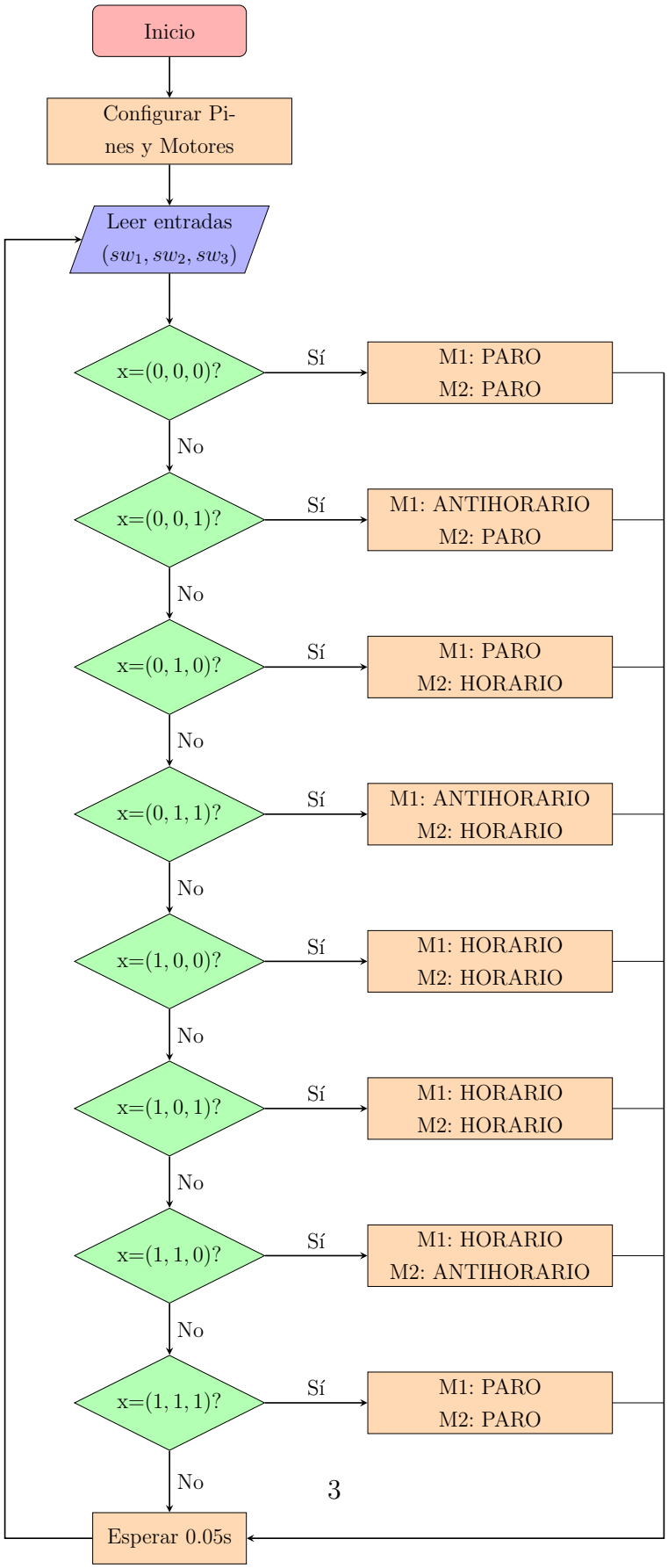
Entradas			Acciones	
SW1_1	**SW1_2**	**SW1_3**	**MOTOR 1**	**MOTOR 2**
GPIO10	*GPIO9*	*GPIO8*	*GPIO0 – GPIO1-GPIO20*	*GPIO2 – GPIO3-GPIO19*
0	0	0	**PARO**	PARO
0	0	1	**GIRA HORARIO**	PARO
0	1	0	**PARO**	GIRA HORARIO
0	1	1	**GIRA HORARIO**	GIRA HORARIO
1	0	0	**GIRA ANTIHORARIO**	GIRA HORARIO
1	0	1	**GIRA HORARIO**	GIRA ANTIHORARIO
1	1	0	**GIRA ANTIHORARIO**	GIRA ANTIHORARIO
1	1	1	**PARO**	PARO

Tabla 1: Tabla 5-2. Controlador; actividad 1

Propuesta de solución

Para controlar de manera independiente dos motores de corriente directa (DC), se propone un algoritmo que interactúa con el driver L293D (un doble puente H). El programa requiere inicializar tres pines de entrada (`GPIO8`, `GPIO9`, `GPIO10`) con resistencias de pull-down físicas o externas para leer el estado del interruptor DIP. Simultáneamente, se configuran seis pines de salida: dos de habilitación (`ENABLE_M1`, `ENABLE_M2`) y cuatro de dirección (`DIR1_M1`, `DIR2_M1`, `DIR1_M2`, `DIR2_M2`).

La lógica central encapsula el control del hardware en una función `control_motor(motor, estado)`. Para inducir movimiento, el pin `ENABLE` correspondiente se satura a 3.3V (nivel lógico alto). El sentido de giro se determina estableciendo una diferencia de potencial en los pines de dirección: un estado alto en `DIR1` y bajo en `DIR2` generará un giro horario, mientras que la configuración inversa generará un giro antihorario. Enviar ceros lógicos a todos los terminales frena el motor. El bucle principal lee las tres entradas físicas como una tupla y la evalúa contra las combinaciones predefinidas en la tabla de verdad para enrutar las señales de voltaje a los pines correspondientes.


Inicio
Configurar Pines y Motores
Leer entradas (sw_1, sw_2, sw_3)
x=(0, 0, 0)?
Sí
M1: PARO
M2: PARO
No
x=(0, 0, 1)?
Sí
M1: ANTIHORARIO
M2: PARO
No
x=(0, 1, 0)?
Sí
M1: PARO
M2: HORARIO
No
x=(0, 1, 1)?
Sí
M1: ANTIHORARIO
M2: HORARIO
No
x=(1, 0, 0)?
Sí
M1: HORARIO
M2: HORARIO
No
x=(1, 0, 1)?
Sí
M1: HORARIO
M2: HORARIO
No
x=(1, 1, 0)?
Sí
M1: HORARIO
M2: ANTIHORARIO
No
x=(1, 1, 1)?
Sí
M1: PARO
M2: PARO
No
Esperar 0.05s

El flujo de datos comienza con la inicialización de los pines: tres entradas digitales conectadas a los interruptores DIP y seis salidas para controlar ambos motores a través del L293D. Una vez configurado el hardware, el programa entra en un ciclo infinito donde primero se captura el estado de los tres switches como una tupla binaria. Esta tupla se compara secuencialmente con las ocho combinaciones posibles definidas en la tabla de verdad. Al encontrar coincidencia, se invoca la función `control_motor()` con los parámetros apropiados, la cual traduce el comando de alto nivel ("HORARIO", "ANTIHORARIO" o "PARO") en señales eléctricas específicas para los pines de habilitación y dirección. Finalmente, se introduce una pausa de 50 milisegundos antes de reiniciar el ciclo, lo cual estabiliza las lecturas y evita oscilaciones por rebote mecánico de los interruptores.

Desarrollo

Listing 1: Código de la Actividad 1

```python
from machine import Pin
import time

# --- 1. CONFIGURACIÓN DE ENTRADAS ---
sw1_1 = Pin(10, Pin.IN)
sw1_2 = Pin(9, Pin.IN)
sw1_3 = Pin(8, Pin.IN)

# --- 2. CONFIGURACIÓN DE SALIDAS (Motores) ---
# Motor 1
dir1_m1 = Pin(0, Pin.OUT)
dir2_m1 = Pin(1, Pin.OUT)
en_m1 = Pin(20, Pin.OUT)

# Motor 2
dir1_m2 = Pin(2, Pin.OUT)
dir2_m2 = Pin(3, Pin.OUT)
en_m2 = Pin(19, Pin.OUT)

# --- 3. FUNCIONES AUXILIARES ---
def control_motor(motor, estado):
    """Controla el giro de un motor específico"""
```

```
    if motor == 1:
        dir1, dir2, en = dir1_m1, dir2_m1, en_m1
    else:
        dir1, dir2, en = dir1_m2, dir2_m2, en_m2

    if estado == "PARO":
        en.value(0)
        dir1.value(0)
        dir2.value(0)
    elif estado == "HORARIO":
        en.value(1)
        dir1.value(1)
        dir2.value(0)
    elif estado == "ANTIHORARIO":
        en.value(1)
        dir1.value(0)
        dir2.value(1)

def leer_entradas():
    return (sw1_1.value(), sw1_2.value(), sw1_3.value())

# --- 4. BUCLE PRINCIPAL ---
while True:
    entradas = leer_entradas()

    # Lógica basada en la Tabla 5-2
    if entradas == (0, 0, 0):
        control_motor(1, "PARO")
        control_motor(2, "PARO")
    elif entradas == (0, 0, 1):
        control_motor(1, "ANTIHORARIO")
        control_motor(2, "PARO")
    elif entradas == (0, 1, 0):
        control_motor(1, "PARO")
        control_motor(2, "HORARIO")
    elif entradas == (0, 1, 1):
```

```
        control_motor(1, "ANTIHORARIO")
        control_motor(2, "HORARIO")
    elif entradas == (1, 0, 0):
        control_motor(1, "HORARIO")
        control_motor(2, "HORARIO")
    elif entradas == (1, 0, 1):
        control_motor(1, "HORARIO")
        control_motor(2, "HORARIO")
    elif entradas == (1, 1, 0):
        control_motor(1, "HORARIO")
        control_motor(2, "ANTIHORARIO")
    elif entradas == (1, 1, 1):
        control_motor(1, "PARO")
        control_motor(2, "PARO")

    time.sleep(0.05) # Estabilizador de lecturas
```

Análisis de resultados

Durante las pruebas en el laboratorio, el sistema respondió correctamente a cada una de las combinaciones de los interruptores. Al colocar los switches en la posición (0, 0, 0), ambos motores permanecieron completamente detenidos, lo cual confirmó que el estado de reposo funciona como se esperaba.

Cuando se activó únicamente el tercer switch, formando la combinación (0, 0, 1), el Motor 1 comenzó a girar en sentido horario mientras el Motor 2 permaneció inmóvil. Este comportamiento demostró que el control individual de cada motor opera de manera independiente, tal como lo requiere la tabla de verdad.

Un aspecto interesante se observó al probar la combinación (1, 1, 1): a pesar de tener todas las entradas en alto, los motores se detienen. Esto tiene sentido desde el punto de vista del diseño, ya que permite usar esta configuración como un “freno de emergencia” o estado seguro del sistema.

El retardo de 50 milisegundos al final de cada ciclo resultó fundamental para eliminar lecturas erráticas causadas por el rebote mecánico de los interruptores. Sin este filtro, los motores presentaban cambios abruptos de dirección incluso cuando los switches estaban fijos.

Actividad 2

De acuerdo a la distribución indicada en el circuito de la figura 5-11; realizar las acciones de control indicadas en la tabla 5-3, considerar la secuencia indicada en la figura 5-4. Conectar la fuente externa con 5V.

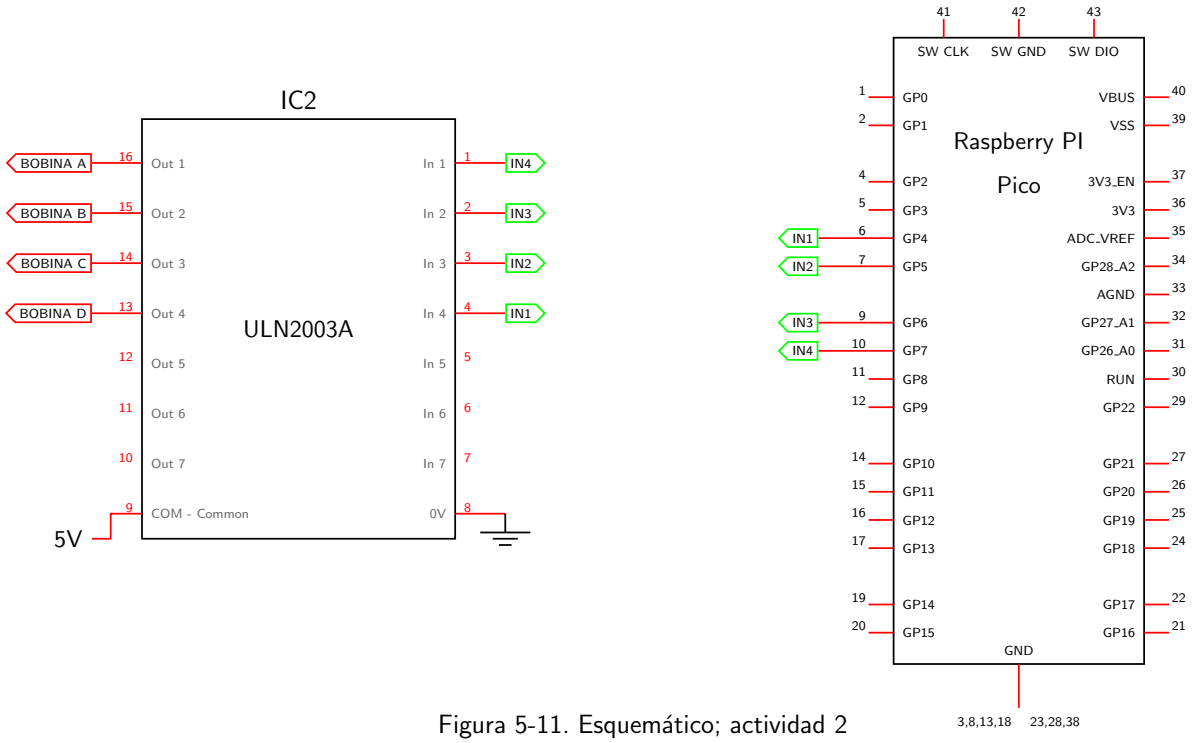


Figura 5-11. Esquemático; actividad 2

Entradas			Acciones
SW1_1	**SW1_2**	**SW1_3**	**Motor a pasos**
GPIO10	*GPIO9*	*GPIO8*	*GPIO7 – GPIO6-GPIO5 – GPIO4*
0	0	0	**PARO**
0	0	1	**GIRA HORARIO**
0	1	0	**GIRA SENTIDO ANTIHORARIO**
0	1	1	**GIRA 90° SENTIDO HORARIO cada 2 seg**
1	0	0	**GIRA 180° SENTIDO ANTIHORARIO cada 3 seg**
1	0	1	**GIRA 360° SENTIDO HORARIO cada 4 seg ***
1	1	0	**GIRA 5 REVOLUCIONES SENTIDO HORARIO ***
1	1	1	**GIRAR 10 REVOLUCIONES SENTIDO ANTIHORARIO ***

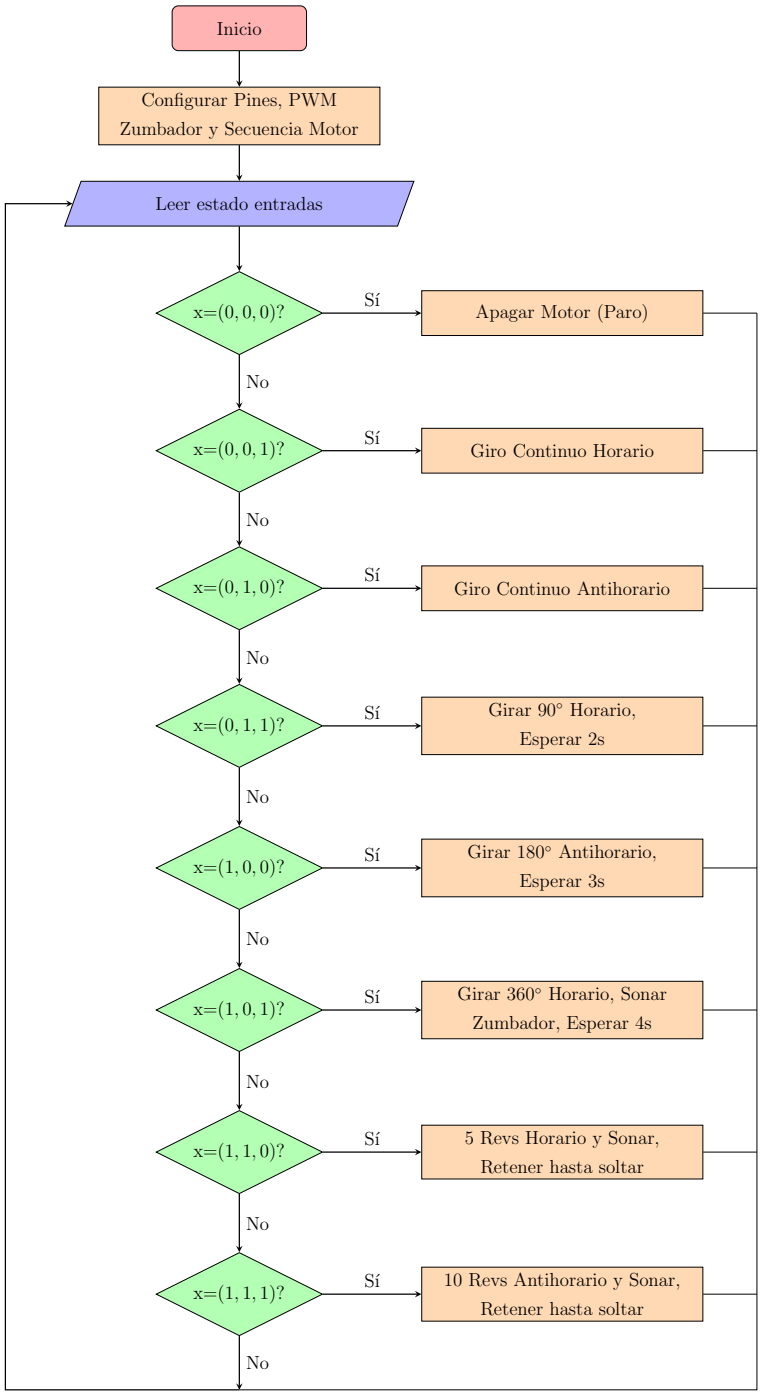
***** *Para los tres últimos casos, cada que se concluya una revolución en el motor a pasos, activar el Zumbador conectado en el GPIO22 durante 300 ms.*

Tabla 2: Tabla 5-2. Controlador; actividad 2

Propuesta de solución

El control de un motor a pasos unipolar requiere energizar sus bobinas en un patrón estricto. La solución implementa una matriz de secuencias de paso completo (`[(1,1,0,0), (0,1,1,0), (0,0,1,1), (1,0,0,1)]`), donde cada fila representa los estados lógicos necesarios en los cuatro pines de salida acoplados al driver Darlington ULN2003A. Se establece una constante `PASOS_POR_REV` (512 o 2048, según la reducción interna del motor) para traducir rotaciones angulares en iteraciones de código.

Se añaden funciones trigonométricas y repetitivas: `girar_grados()` divide el ángulo objetivo entre 360 y lo multiplica por los pasos por revolución para determinar el número exacto de cambios de fase requeridos; `girar_revoluciones()` encapsula rotaciones completas. Adicionalmente, se integra retroalimentación acústica configurando el GPIO 22 con Modulación por Ancho de Pulso (PWM). Al finalizar un ciclo específico, el programa activa el PWM a una frecuencia audible para excitar un zumbador pasivo. Las condicionales del bucle principal leen la tupla de interruptores y dirigen el flujo hacia movimientos continuos, pasos angulares o revoluciones completas con temporizaciones específicas.



El flujo de datos de la Actividad 2 comienza con la configuración inicial de los puertos GPIO: tres pines como entrada para el interruptor DIP, cuatro pines de salida destinados a las bobinas del motor y un pin con PWM configurado para el zumbador. Además, se define una matriz con la secuencia lógica de excitación de las bobinas en paso completo.

Durante el ciclo principal infinito, se leen constantemente los tres interruptores para formar una tupla representativa del estado actual. A través de una cascada de decisiones lógicas, se direcciona el programa a los métodos específicos correspondientes a cada acción. Dependiendo del caso, el sistema ejecuta rutinas de paro, movimiento continuo iterando el estado de paso, o movimientos controlados haciendo uso de las funciones de grados o revoluciones. Para las funciones controladas, el flujo invoca funciones auxiliares que iteran de forma precisa sobre los pasos exactos requeridos, activando después la señal auditiva del zumbador. Finalizado cualquier bloque de acción, se reinicia el flujo tomando la nueva lectura de hardware.

Desarrollo

Listing 2: Código de la Actividad 2

```
from machine import Pin, PWM
import time

# --- ENTRADAS ---
sw1_1 = Pin(10, Pin.IN)
sw1_2 = Pin(9, Pin.IN)
sw1_3 = Pin(8, Pin.IN)

# --- SALIDAS ---
bobinas = [Pin(7, Pin.OUT), Pin(6, Pin.OUT), Pin(5, Pin.OUT), Pin(4,
    Pin.OUT)]
buzzer = PWM(Pin(22))
buzzer.duty_u16(0)

# --- CONFIGURACIÓN DEL MOTOR ---
# Secuencia de pasos completos (A, B, C, D)
secuencia = [(1,1,0,0), (0,1,1,0), (0,0,1,1), (1,0,0,1)]
PASOS_POR_REV = 512  # Ajusta a 2048 si usas un motor 28BYJ-48 con
   reducción completa
TIEMPO_PASO = 0.005  # 5 ms de velocidad por paso

def leer_entradas():
    return (sw1_1.value(), sw1_2.value(), sw1_3.value())

```

```python
def apagar_motor():
    for b in bobinas: b.value(0)

def dar_paso(idx):
    for i in range(4): bobinas[i].value(secuencia[idx][i])

def sonar_zumbador():
    buzzer.freq(1000)
    buzzer.duty_u16(32768)
    time.sleep(0.3)
    buzzer.duty_u16(0)

def girar_grados(grados, sentido, sonar=False):
    pasos = int((grados / 360.0) * PASOS_POR_REV)
    idx = 0
    for _ in range(pasos):
        if leer_entradas() not in [(0,1,1), (1,0,0), (1,0,1),
           (1,1,0), (1,1,1)]: break
        dar_paso(idx)
        time.sleep(TIEMPO_PASO)
        idx = (idx + 1) if sentido == "HORARIO" else (idx - 1)
        idx %= 4
    apagar_motor()
    if sonar: sonar_zumbador()

def girar_revoluciones(revs, sentido, sonar_por_rev=True,
   condicion_salida=None):
    for _ in range(revs):
        if leer_entradas() != condicion_salida: break
        girar_grados(360, sentido, sonar=False)
        girar_grados(360, sentido, sonar=False)
        girar_grados(360, sentido, sonar=False)
        girar_grados(360, sentido, sonar=False)
        if sonar_por_rev: sonar_zumbador()

idx_continuo = 0
```

```python

while True:
    estado = leer_entradas()

    if estado == (0, 0, 0): # PARO
        apagar_motor()

    elif estado == (0, 0, 1): # HORARIO CONTINUO
        dar_paso(idx_continuo)
        time.sleep(TIEMPO_PASO)
        idx_continuo = (idx_continuo + 1) % 4

    elif estado == (0, 1, 0): # ANTIHORARIO CONTINUO
        dar_paso(idx_continuo)
        time.sleep(TIEMPO_PASO)
        idx_continuo = (idx_continuo - 1) % 4

    elif estado == (0, 1, 1): # 90° HORARIO cada 2 seg
        girar_grados(90, "HORARIO")
        time.sleep(2)

    elif estado == (1, 0, 0): # 180° ANTIHORARIO cada 3 seg
        girar_grados(360, "ANTIHORARIO")
        girar_grados(360, "ANTIHORARIO")
        time.sleep(3)

    elif estado == (1, 0, 1): # 360° HORARIO cada 4 seg + Zumbador
        girar_grados(360, "HORARIO")
        girar_grados(360, "HORARIO")
        girar_grados(360, "HORARIO")
        girar_grados(360, "HORARIO", sonar=True)
        time.sleep(4)

    elif estado == (1, 1, 0): # 5 REVS HORARIO + Zumbador x Rev
        girar_revoluciones(5, "HORARIO", True, (1, 1, 0))
        while leer_entradas() == (1, 1, 0): time.sleep(0.1) # Evita
```

```
            bucle infinito si no sueltas

    elif estado == (1, 1, 1): # 10 REVS ANTIHORARIO + Zumbador x Rev
        girar_revoluciones(10, "ANTIHORARIO", True, (1, 1, 1))
        while leer_entradas() == (1, 1, 1): time.sleep(0.1)
```

Análisis de resultados

Las pruebas con el motor a pasos mostraron un control preciso del movimiento en todos los modos de operación. Con la combinación (0, 0, 1), el motor giró de forma continua en sentido horario, y al cambiar a (0, 1, 0), invirtió su dirección inmediatamente sin necesidad de detenerse primero.

Los movimientos angulares resultaron particularmente interesantes. Al probar el modo de 90 grados, el motor completó exactamente un cuarto de vuelta y se detuvo. Calculando manualmente, 90 grados corresponden a 128 pasos (512 pasos por revolución divididos entre 4), y efectivamente el eje se posicionó en el ángulo esperado cada vez.

El zumbador funcionó como indicador audible en los modos que lo requerían. Durante la prueba de 360 grados con la combinación (1, 0, 1), se escuchó claramente el tono de 1 kHz al completar cada revolución. Este tipo de retroalimentación resulta muy útil para confirmar visualmente que el programa terminó su tarea, especialmente cuando el movimiento del motor es difícil de distinguir a simple vista.

Un detalle importante fue el bloqueo implementado en los modos de múltiples revoluciones. Al activar la combinación (1, 1, 0) para las 5 revoluciones, el programa ejecutó todo el movimiento y después esperó a que se cambiaran los switches antes de repetir. Sin este mecanismo, el motor habría iniciado nuevamente las 5 vueltas de forma infinita.

Actividad 3

De acuerdo a la distribución indicada en el circuito de la figura 5-12; realizar las acciones de control indicadas en la tabla 5-4, considerar la información indicada en la figura 5-7. Conectar la fuente externa con 5V.

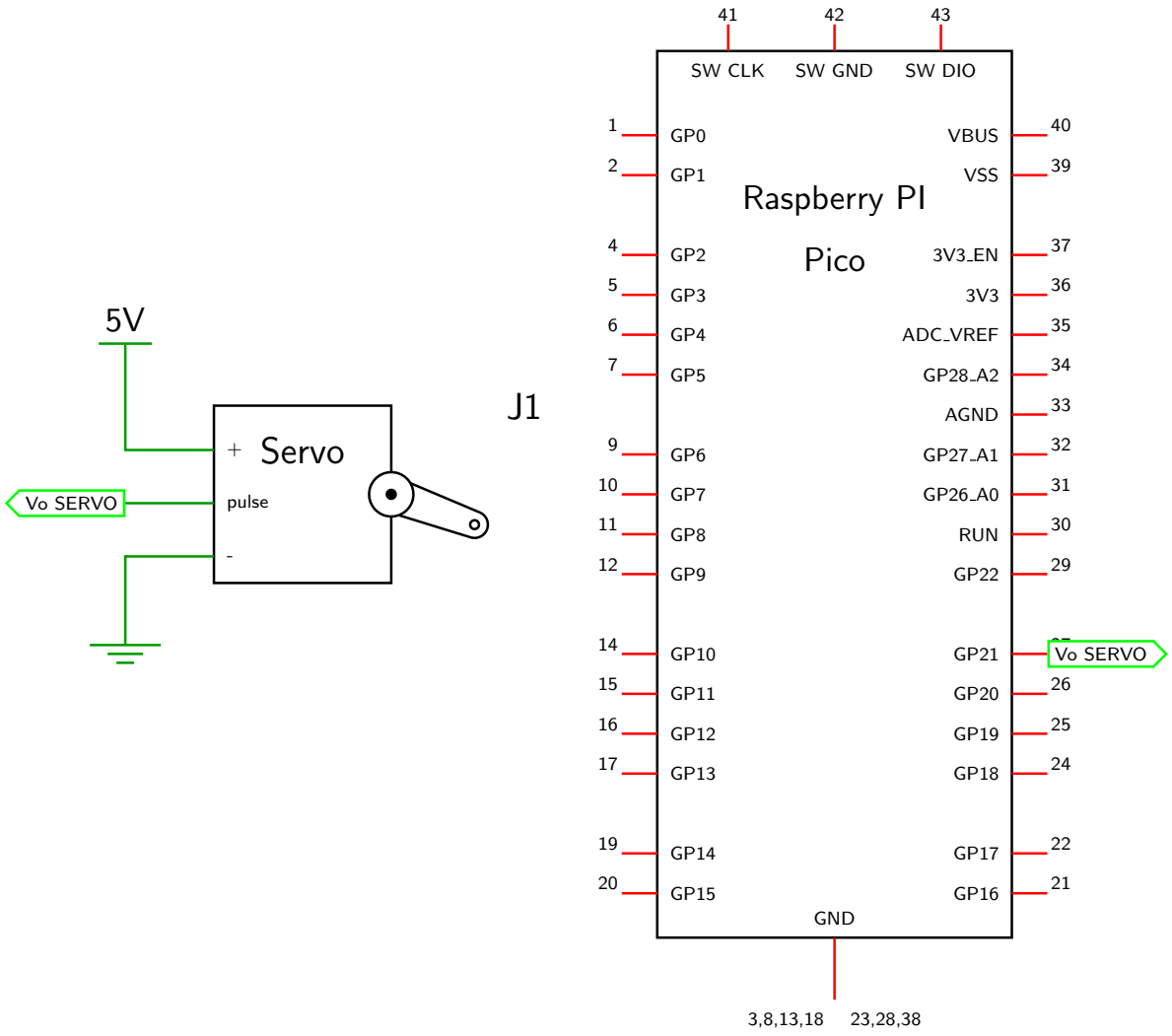


Figura 5-12. Esquemático control del servomotor

Entradas					Acciones
S2 *GPIO12*	**S1** *GPIO11*	**SW1_1** *GPIO10*	**SW1_2** *GPIO9*	**SW1_3** *GPIO8*	**SERVOMOTOR** *GPIO21*
0	1	0	0	0	**PARO**
0	1	0	0	1	**SERVO A 0°**
0	1	0	1	0	**SERVO A 45°**
0	1	1	0	0	**SERVO A 90°**
0	0	0	0	0	**SERVO A 135°**
1	1	0	0	0	**SERVO A 180°**

Tabla 3: Tabla 5-3. Controlador; actividad 3

Propuesta de solución

Desarrollo

Listing 3: Código de la Actividad 3

```
from machine import Pin, PWM
import time

# --- ENTRADAS ---
S2 = Pin(12, Pin.IN, Pin.PULL_DOWN)
S1 = Pin(11, Pin.IN, Pin.PULL_UP)
sw1_1 = Pin(10, Pin.IN)
sw1_2 = Pin(9, Pin.IN)
sw1_3 = Pin(8, Pin.IN)

# --- SALIDA PWM (Servo) ---
servo = PWM(Pin(21))
servo.freq(50) # El estándar para servomotores es un periodo de 20ms
    (50Hz)

def leer_entradas():
    return (S2.value(), S1.value(), sw1_1.value(), sw1_2.value(),
```

```
        sw1_3.value())

def set_servo_angulo(angulo):
    """
    Convierte un ángulo de 0 a 180 en el ciclo de trabajo
        equivalente.
    0.5ms (1638 duty) equivale a 0°.
    2.5ms (8192 duty) equivale a 180°.
    """
    min_duty = 1638
    max_duty = 8192
    duty = int(min_duty + (angulo / 180.0) * (max_duty - min_duty))
    servo.duty_u16(duty)

while True:
    estado = leer_entradas()
    if estado == (0, 1, 0, 0, 0):
        servo.duty_u16(0) # PARO (Apaga la señal PWM, liberando el
            torque del servo)
    elif estado == (0, 1, 0, 0, 1):
        print(estado)
        set_servo_angulo(0)
    elif estado == (0, 1, 0, 1, 0):
        print(estado)
        set_servo_angulo(45)
    elif estado == (0, 1, 1, 0, 0):
        print(estado)
        set_servo_angulo(90)
    elif estado == (0, 0, 0, 0, 0):
        print(estado)
        set_servo_angulo(135)
    elif estado == (1, 1, 0, 0, 0):
        print(estado)
        set_servo_angulo(180)

    time.sleep(0.1)
```

Análisis de resultados

Actividad 4

Realizar un programa, de manera que controle la posición del servo de forma automática, iniciando en la posición de 0o hasta $180^{\underline{o}}$ y viceversa.

Propuesta de solución

Desarrollo

Listing 4: Código de la Actividad 4

```python
from machine import Pin, PWM
import time

# --- CONFIGURACIÓN ---
servo = PWM(Pin(21))
servo.freq(50) # 50 Hz = 20 ms de periodo

def set_servo_angulo(angulo):
    min_duty = 1638
    max_duty = 8192
    duty = int(min_duty + (angulo / 180.0) * (max_duty - min_duty))
    servo.duty_u16(duty)

# --- BUCLE PRINCIPAL ---
while True:
    # Barrido ascendente (0° a 180°)
    for angulo in range(0, 181, 5): # Avanza de 5 en 5 grados
        set_servo_angulo(angulo)
        time.sleep(0.05) # Velocidad del barrido

    # Barrido descendente (180° a 0°)
    for angulo in range(180, -1, -5):
        set_servo_angulo(angulo)
        time.sleep(0.05)
```

Análisis de resultados

2. Conclusiones:

- **Espinoza Matamoros Percival Ulises:**
- **Flores Colin Victor Jaziel:**
- **Lara Hernandez Angel Husiel:**

Referencias

Arm Ltd. (2020). *Arm Cortex-M0+ Processor Datasheet* (Datasheet) (Documento PDF. Año aproximado). Arm Limited.

Gómez, A. (2015, agosto). *Resistencia pull up y pull down con Arduino, para qué sirven.* Consultado el 27 de febrero de 2026, desde https://programarfacil.com/blog/arduino-blog/resistencia-pull-up-y-pull-down/ Blog de tecnología y programación.

Raspberry Pi Ltd. (2025, 20 de febrero). *RP2040 Datasheet: A microcontroller by Raspberry Pi.* Ver. build-version: 3184e62-clean. Consultado el 27 de febrero de 2026, desde https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf

Raspberry Pi Ltd. (2026). *Microcontroller chips - Raspberry Pi Documentation.* Consultado el 26 de febrero de 2026, desde https://www.raspberrypi.com/documentation/microcontrollers/microcontroller-chips.html

Raspberry Pi Stack Exchange Users. (2021, diciembre). *Why is a resistor needed for LEDs?* Consultado el 27 de febrero de 2026, desde https://raspberrypi.stackexchange.com/questions/4357/why-is-a-resistor-needed-for-leds Foro de Q&A. Colaboradores: Steve Robillard, joan, Jivad.

Transfer Multisort Elektronik (TME). (2024). *GPIO - definición — Glosario TME.* Consultado el 27 de febrero de 2026, desde https://www.tme.com/mx/es/news/library-articles/glossary/page/61888/gpio-general-purpose-inputoutput-definicion/ Artículo de glosario técnico.